```
class Formulaic
  toJSON: -> "message": "Determine your own JSON representation"

formulaic = new Formulaic
JSON.stringify formulaic
# '{"message":"Determine your own JSON representation"}'
```

That takes care of sending the form information to the server. How about getting the form back? Suppose the server responds to a POST with updated JSON for the object:

```
sync: ->
  throw new Error 'No transport' unless @http? and @url?
  @http.post @url, JSON.stringify(@extractFields()), (response) ->
    @fields = JSON.parse response
```

What does invoking `JSON.parse` do?

parse

The `JSON.parse` method converts a string of JSON to an object. If you need to polyfill `JSON.parse`, you can do it dangerously with an `eval`:

```
JSON.parse ?= (json) ->
  eval json

barney = JSON.parse '{"firstName":"Barney","lastName":"Rubble"}'"
barney.lastName
# "Rubble"
```

Be warned: `eval` is evil. When you `eval` code, it can do anything, so you should never `eval` code you don't trust. The ECMAScript 5 specification adds a native `JSON.parse` over `eval` (for safety reasons) and over other techniques for parsing JSON (for performance reasons).

The native JSON support is the most immediately applicable solution for Formulaic. How does Formulaic send form data back to the server and what format does it use? In the next listing you see an extended version of Formulaic that uses native JSON support.

Listing 13.2. Formulaic with server sync